kernoid

kernoid: One Sovereign Core for Email, Identity and Storage

Whitepaper v0.2 (draft) · October 2026

Executive Summary

Email and identity are the core of every organisation's IT – and the core of its dependency. Whoever runs an organisation's mailboxes and its accounts also runs the logins to every other service, the calendars, the contacts and, usually, the files. Many organisations have handed that core to Microsoft or Google – universities in the United States, the United Kingdom and the Netherlands largely, those in Germany, France, Austria and Switzerland far less[1]. Switching away is hard not because good free software is missing, but because the free building blocks are separate systems: one for mail, one for identity, one for files – each with its own users, its own administration and its own operations.

kernoid is one system for that core. kernoid is a self-hostable cloud application, written in Rust, that combines an email and collaboration server with an identity provider and storage – as one program with one data model:

- **one identity** – an account is created once and can be at the same time the mailbox, the login for single sign-on and the owner of files; nothing has to be synchronised;
- **one administration** – users, groups, domains and policies are managed in one place;
- **one operation** – one application to deploy, update and monitor, built for containers and Kubernetes, with PostgreSQL for data and S3-compatible object storage for messages and files.

Built on proven free software. kernoid does not start from zero. It takes over what it can from two mature projects under the GNU AGPL-3.0: from **Stalwart**, a mail and collaboration server written in Rust (SMTP, IMAP, JMAP, CalDAV, CardDAV, WebDAV)[2], and from **Zitadel**, an identity provider with OpenID Connect, SAML 2.0, SCIM 2.0 and passkeys[3]. kernoid reuses Stalwart's AGPL-licensed code where it fits and carries Zitadel's concepts and protocol logic over to Rust – with the attribution both licences require.

Everything included, for everyone. kernoid is free software under the GNU AGPL-3.0, and the free release is the full product: single sign-on, provisioning, multi-tenancy and clustering are part of it – the very features that other projects reserve for a paid edition; Stalwart, for example, lists SCIM provisioning and multi-tenancy with per-tenant identity backends as enterprise features[4]. kernoid collects no telemetry.

[1] Fiebig et al., "Heads in the Clouds? Measuring Universities' Migration to Public Clouds", Proceedings on Privacy Enhancing Technologies 2023(2) (MX records of universities), checked 2026-10-22: https://arxiv.org/abs/2104.09462

[2] Stalwart, README of `stalwartlabs/stalwart` (AGPL-3.0 and SELv2), checked 2026-10-22: https://github.com/stalwartlabs/stalwart

[3] Zitadel, README of `zitadel/zitadel` (AGPL-3.0), checked 2026-10-22: https://github.com/zitadel/zitadel

[4] Stalwart Enterprise, feature list, checked 2026-10-22: https://stalw.art/enterprise/

A falkOSS project. kernoid is published by falkOSS, a non-profit company for free software in formation in Falkensee near Berlin **[OPEN: status of the incorporation and of the non-profit recognition at publication]**. falkOSS holds the name and funds the development; the technical direction lies with the project. As soon as kernoid is stable, falkOSS will run its own email on it.

The Problem

Whoever runs the core holds the keys

Every other service of an organisation hangs on two things: the **login**, which single sign-on hands out for all applications, and the **email address**, to which every account outside single sign-on sends its password resets. Whoever operates mail and identity can let people in and lock them out – everywhere at once.

That is not a theoretical risk. Providers based in the United States must hand over data on request "regardless of whether such communication, record, or other information is located within or outside of the United States"[5] – a European data centre does not change that. And in May 2025 the chief prosecutor of the International Criminal Court lost access to his Microsoft email after the US sanctioned the court; Microsoft states that it never stopped its services to the court itself[6]. Whatever happened in detail: an organisation that does not run its core itself does not decide alone who keeps access to it.

Free building blocks, separate systems

There is good free software for every part of the core – mail servers, identity providers, file services. But each is its own system with its own accounts. A self-hosted setup typically connects them: the mail server reads its users from a directory over LDAP, accepts logins from an identity provider over OpenID Connect, or has its accounts created and removed by that identity provider over SCIM. Stalwart, for example, offers all three ways next to its internal directory[7].

Connected systems are not one system:

- **An account exists several times** – in the identity provider, in the mail server, in the file service – and every copy has to be created, changed and removed. If one connection fails, a person who left keeps a working mailbox, or a new colleague has none.
- **Rights live in several places.** Who may read a shared mailbox, who may open a folder, who is administrator: each system answers that itself.
- **Operation multiplies:** several administration interfaces, several update cycles, several backups, several sets of logs to look at when something goes wrong.

That is not a weakness of the individual projects. It follows from building the core out of separate systems.

The connecting parts cost extra

Many free projects finance themselves through a paid edition – a legitimate model. But what moves into the paid edition is often exactly what connects systems or serves many organisations at once:

- Stalwart lists SCIM provisioning and multi-tenancy with per-tenant identity backends as enterprise features[8].

[5] 18 U.S.C. § 2713, added by the CLOUD Act (Pub. L. 115–141, div. V, 2018), checked 2026-10-22: https://www.law.cornell.edu/uscode/text/18/2713

[6] heise online, "Criminal Court: Microsoft's email block a wake-up call for digital sovereignty", 19 May 2025: https://heise.de/-10387383; heise online, "Microsoft denies mail blockade at the International Criminal Court", 5 June 2025, checked 2026-10-22: https://heise.de/-10429628

[7] Stalwart documentation, authentication backends (internal; external LDAP, SQL, OIDC; SCIM provisioning), checked 2026-10-22: https://stalw.art/docs/auth/backend/overview

– authentik reserves provisioning to Google Workspace and Microsoft Entra ID, among others, for its enterprise edition[9].

An organisation that wants to run its core seriously ends up paying in several places – or living without these features.

Mail has to arrive

A mail server can be configured perfectly and its messages still land in the spam folder. The large mailbox providers set the rules: since February 2024 Google requires SPF or DKIM, valid reverse DNS and TLS from every sender, and from bulk senders also DMARC and one-click unsubscribe[10]; Microsoft applies similar rules to domains that send 5,000 or more messages to Outlook.com[11]. Beyond those rules, a new server's address has no sending history – and reputation cannot be configured.

kernoid makes the configurable part the default and checks it: it shows when SPF, DKIM, DMARC, reverse DNS or MTA-STS are missing or wrong. Reputation it cannot create – but a new installation can send through a relay of the operator's choice until its own address has built a history.

Positioning

What kernoid is

kernoid is **one application** for an organisation's core services:

Area	Scope
Email	SMTP (in and out), IMAP, JMAP; spam and phishing filtering; SPF, DKIM, DMARC, MTA-STS, DANE
Collaboration	calendars and contacts (CalDAV, CardDAV, JMAP), sharing
Storage	files with sync, sharing and versions – kernoid's own storage, with first-party apps (see below)
Identity	accounts, groups, roles; OpenID Connect, OAuth 2.0, SAML 2.0, SCIM 2.0, LDAP; MFA and passkeys
Administration	one interface and one API for all of the above, multi-tenancy
Operation	self-hosted on Docker or Kubernetes; needs only PostgreSQL and S3-compatible object storage

Mailbox, login and file owner are **one object** – an account that is whichever of them it needs to be. That is the difference to running a mail server and an identity provider side by side and connecting them.

Everything included

The free release is the full product. Single sign-on, provisioning, multi-tenancy, clustering, audit logs – everything an organisation needs to run its core seriously – is part of every kernoid release, for everyone. There is one edition, and it is free.

kernoid is funded through falkOSS – by donations, grants and the service providers who build on it – not by holding features back (see Governance and Funding).

[8] Stalwart Enterprise, feature list, checked 2026-10-22: https://stalw.art/enterprise/

[9] authentik documentation, enterprise features, checked 2026-10-22: https://docs.goauthentik.io/enterprise/enterprise-features/

[10] Google Workspace Admin Help, "Email sender guidelines", checked 2026-10-22: https://support.google.com/a/answer/81126

[11] Microsoft Support, "Fix NDR error 550 5.7.515 in Outlook.com", checked 2026-10-22: https://support.microsoft.com/en-us/outlook/fix-ndr-error-550-5-7-515-in-outlook-com

Apps and protocol

kernoid comes with **first-party apps**: the web app and the terminal app, and – with the first stable release – one app for desktop and mobile, built with Flutter for macOS, iOS, Linux, Android and Windows. The web app is a progressive web app written in TypeScript; with it comes a **TypeScript SDK**, so that anyone can build kernoid's login, or more, into their own web application. The terminal app and the desktop and mobile app build on a **Rust SDK**: the protocol, file sync and end-to-end encryption, published as libraries anyone can use. End-to-end encryption is a focus of the apps: what one sets up by hand in other mail programs, they do by themselves.

The apps speak an **open, documented protocol**; anyone can write their own apps and clients against it, and existing clients keep working through the standard protocols (IMAP, JMAP, CalDAV, CardDAV, WebDAV).

For file sync, kernoid adopts an existing open protocol: **JMAP for files** (`FileNode`, an IETF draft), so that one protocol family serves mail, calendars, contacts and files; WebDAV serves the programs that speak it.

The licences are split on purpose:

Part	Licence	Why
Server (kernoid)	GNU AGPL-3.0-or-later; code taken over from Stalwart and Zitadel stays AGPL-3.0-only	whoever offers kernoid as a service must share their changes
Terminal app	GNU AGPL-3.0-or-later	it lives with the server and builds on the MIT libraries
Web, desktop and mobile apps, client libraries (SDKs)	MIT	apps can be distributed through app stores; anyone can build on them
Protocol specification	CC BY 4.0, freely implementable	no client depends on kernoid's own apps

The apps are decoupled from the server: they share no AGPL code with it, only the protocol. Code taken over from Stalwart or Zitadel therefore stays in the server. The terminal app is under the server's licence, but it too uses only the MIT libraries, not the server's code.

DRAFT

No lock-in, not even to kernoid

kernoid speaks the open standards in both directions: mail over IMAP and JMAP, calendars and contacts over CalDAV and CardDAV, files over WebDAV, accounts over SCIM and LDAP. The same protocols that bring an organisation's data into kernoid take it out again. An organisation that leaves kernoid takes its data with it, in standard formats.

Where it comes from

kernoid stands on the work of others and says so: it takes over **Stalwart**'s AGPL code for mail and collaboration and carries **Zitadel**'s concepts and logic for identity over to Rust – see Building on Stalwart and Zitadel.

The name

```
Kern    German      → core
kerno   Esperanto   → core
kerno + id          → kernoid
kern0id, 0 as o     → kerno.id
kern0id, without 0  → kern.id
kernoid, OIDC       → OpenID Connect
```

Kern is German for core, and *kerno* is Esperanto for core. Both are in the wordmark `kern0id`: read the 0 as an o and it says **kerno.id**, leave it out and it says **kern.id** – the core, with identity; the dot in the 0 is the dot of the domain. kernoid joins the two: *kerno* and *id*. The *oid* also winks at OpenID Connect (OIDC), the protocol kernoid logs people in with, and, as an aside, at mathematics: the kernel of a map, and *-oid* as in monoid – kernel and monoid make kernoid.

What kernoid is not

- **Not an integration layer** on top of separate servers – it is one system.
- **Not an office suite.** kernoid builds where integration is the problem – the core – and connects where integration already works: documents, video and chat come from proven free software that logs in through kernoid (see Integrations).
- **Not a hosting company.** kernoid is software that anyone can run. Managed hosting is offered by others: by providers of the operator's choice, and planned by a separate company **[OPEN: name, timing]**. falkOSS itself hosts kernoid only for non-profit purposes.

Why Rust

kernoid handles the most sensitive data an organisation has: its mail and its accounts. Rust removes whole classes of memory-safety bugs, delivers the performance needed to serve many tenants from few machines, and lets kernoid build directly on Stalwart's existing Rust code.

Architecture

One data model

kernoid is built around a few objects that every part of the system shares:

- An **instance** is one identity provider with its own domains and its own administrators; it holds tenants. An installation usually runs one instance – several serve a managed service or an umbrella organisation without separate deployments.
- A **tenant** is an organisation: its domains, its policies, its administrators. An account's address names its tenant: `alice@example.org` belongs to the tenant that owns `example.org`.
- An **account** belongs to a tenant and can be at the same time a login, a mailbox, the owner of calendars and contacts and the owner of files – whichever it needs; the tenant decides who gets which.
- A **group** collects accounts and is at the same time a mailing list and the one to give rights to on mailboxes, calendars and folders – one object instead of three.
- A **shared mailbox** has an address but no person behind it (info@, support@); rights decide who works with it.
- A **drive** holds folders that belong to the tenant, not to a person; accounts and groups get rights to them.
- A **resource** – a room, a device – has an address and a calendar and can be booked **[OPEN: not built yet]**.
- A **service account** belongs to a machine or an application and uses keys instead of a login **[OPEN: not built yet]**.
- **Roles** say what an account or a group may do – one rights model for mailboxes, calendars, folders and administration.
- **Applications** are the other services of a tenant that log in through kernoid over OpenID Connect or SAML.

Because these objects exist once, there is nothing to synchronise inside kernoid: an account that is disabled can no longer log in, receive mail or open files – in one step.

DRAFT

Components

Component	Does	Protocols
Identity and auth	accounts, groups, roles, login, MFA and passkeys, single sign-on for other applications	OpenID Connect, OAuth 2.0, SAML 2.0, SCIM 2.0, LDAP
Mail	receiving, filtering, storing and sending mail	SMTP, IMAP, JMAP
Collaboration	calendars, contacts, sharing	CalDAV, CardDAV, JMAP
Storage and sync	files, folders, versions, sharing, sync with the apps	JMAP for files, WebDAV
Administration	one interface and one API for all of the above	REST (described with OpenAPI) and GraphQL, over HTTPS with JSON

All components are parts of one program, not separate services: they use the same objects and the same rights model directly.

Modules

The core – identity, mail, collaboration, storage – is always there. Everything that connects kernoid to a particular outside service is a **module**: video meetings with Jitsi, editing documents with office software over WOPI, virus scanning, external key management, DNS automation, moving over from other services, SCIM connections to specific services – and further modules as integrations are added (see Integrations; for the order, see Roadmap).

- Modules are maintained by the kernoid project in its own repository and **compiled into the one image**: they are tested and released together with the core, and no code is loaded at run time.
- Modules are **off by default** and are switched on where they are needed – per installation, and per tenant.
- A module that is off does nothing: it opens no ports and runs no background jobs.
- Modules are part of the same free release. They are not add-ons that cost extra – *everything included* applies to them as well.

A service that speaks an open standard needs no module: whatever logs in over OpenID Connect or takes accounts over SCIM works with kernoid as it is. Everyone else extends kernoid from outside, with actions and hooks (see Integrations).

Data

kernoid needs **PostgreSQL** and **S3-compatible object storage** – nothing else:

- **Identity and administration** are stored as events, following Zitadel, which is built on the event-sourcing pattern and gets its audit trail from it[12]. Every change to an account, a role or a policy is an event; the current state is derived from them. The audit log is not an extra feature, it is the data.
- **Mail, calendars and contacts:** structure and metadata in PostgreSQL, message bodies and attachments in object storage.
- **Files:** folders, versions and rights in PostgreSQL, content in object storage.
- **Search and coordination** between instances run on PostgreSQL as well; an external search index can be added later as an option for large installations, but is never required.

[12] Zitadel documentation, "Event store", checked 2026-10-22: https://zitadel.com/docs/concepts/eventstore/overview

DRAFT

Operation

kernoid ships as **one container image**. Configuration decides which roles a process takes on: in a small installation one process does everything; in a large one, receiving mail, IMAP and JMAP, the web interface and API, and background jobs run as separate, independently scaled groups of processes. The processes keep no state of their own – everything lives in PostgreSQL and object storage – so they can be added, removed and updated one after another.

kernoid comes with **Docker Compose** files for small installations and a **Kubernetes operator** with a Helm chart for large ones, which also runs the database migrations on upgrades **[OPEN: backups through the operator; upgrade policy and supported versions]**.

Security

- **Memory safety:** written in Rust, see Why Rust.
- **In transit:** TLS for every protocol, MTA-STS and DANE for mail between servers. Every TLS connection prefers a key exchange that quantum computers cannot break later (ML-KEM together with X25519), and falls back to X25519 for the other side's sake. kernoid terminates TLS itself or runs behind a reverse proxy such as Caddy, nginx or Traefik; it believes forwarded headers only from proxies it is configured to trust.
- **At rest:** kernoid encrypts message and file content itself, with a key per tenant, before it goes to object storage – the storage provider only ever sees ciphertext. PostgreSQL is encrypted by the storage layer below it. The tenant keys are protected by a main key from the configuration, or held in an external key store through a module (see Modules). Both kinds of keys can be rotated: new content uses the new tenant key while older content stays readable, and a new main key wraps all tenant keys again.
- **Login:** MFA and passkeys for every account; policies per tenant.
- **Traceability:** every change to identity and administration is an event (see Data).
- **End-to-end encryption:** mail, folders, calendars and contacts that only their owners can read – not the operator, not kernoid – with OpenPGP. It is a setting, because other software cannot read what is encrypted; in kernoid's own apps it simply works – in the web app and the terminal app since 0.2, in the app for desktop and mobile from its first version. Only code the user does not load from the server protects against the operator, so the native apps matter for it.
- **Review:** an external security audit takes place before 1.0, and its results are published **[OPEN: funding of the audit]**.

Building on Stalwart and Zitadel

kernoid does not start from zero. Two mature free projects have solved large parts of what it needs: Stalwart for mail and collaboration, Zitadel for identity. This chapter says what kernoid takes from them, under which rules, and how it deals with both projects.

Stalwart: code

Stalwart is a mail and collaboration server written in Rust[13]. kernoid takes over Stalwart code where it fits – above all the protocols (SMTP, IMAP, JMAP, CalDAV, CardDAV, WebDAV) and mail filtering. kernoid copies the selected parts **once** into its own repository, removes everything under the enterprise licence and adapts the code to its own data model; it does not merge Stalwart's later changes. So far kernoid has taken over the protocol parsers for IMAP and for WebDAV, CalDAV and CardDAV, DANE, and parts of

[13] Stalwart, README of `stalwartlabs/stalwart` (AGPL-3.0 and SELv2), checked 2026-10-22: https://github.com/stalwartlabs/stalwart

the calendar logic (filters, free-busy); Stalwart's libraries for parsing, authenticating, building and sending mail and for calendars and contacts are dependencies under Apache-2.0 or MIT.

Stalwart is published under two licences, and every source file says which applies[14]:

Files (Rust, `main`, 2026-10-22)	Licence	kernoid
907	`AGPL-3.0-only OR LicenseRef-SEL` – free choice	takes them under the AGPL-3.0
102	`LicenseRef-SEL` only (Stalwart Enterprise License)	takes nothing – among them all of SCIM
52 of the 907	contain sections marked `SPDX-SnippetBegin` … `SPDX-SnippetEnd` under `LicenseRef-SEL`	removes these sections

What Stalwart reserves for its enterprise edition – SCIM provisioning, parts of multi-tenancy – kernoid writes itself. An automated check keeps the rule: no file and no section under `LicenseRef-SEL` ever enters kernoid's repository – every change is checked for it automatically before it is merged.

Zitadel: concepts and logic

Zitadel is an identity provider written in Go, under AGPL-3.0-only; some parts are under Apache-2.0 (API definitions, documentation) or MIT (the login application, client packages)[15]. kernoid carries over Zitadel's event model, its multi-tenancy, its projects and project grants, its login names and the linking of external logins, and the logic of its protocols – OpenID Connect, OAuth 2.0, SAML 2.0, SCIM 2.0 – and rewrites them in Rust.

A translation from Go to Rust is still a derivative work. kernoid therefore draws a clear line: **code written while reading Zitadel's source is marked as derived from Zitadel** and keeps its licence and copyright notice. What kernoid builds only from specifications and documentation is its own code.

kernoid's login page is its own, written for kernoid; it takes nothing from Zitadel's login application.

Licences in one repository

- **kernoid's own code** is under the AGPL-3.0-or-later.
- **Code taken over** from Stalwart or derived from Zitadel keeps the AGPL-3.0-only, its copyright notice and a note of its origin.
- **The server as a whole** is therefore distributed under the AGPL-3.0. A move to a later version of the AGPL would only be possible once the taken-over code is replaced.
- **The apps are not affected:** they share no code with the server (see Apps and protocol).

Every file names its licence and its authors in SPDX headers; a credits file (CREDITS.md) lists both projects and what kernoid took from them. kernoid follows the REUSE specification, so every file's licence and copyright can be checked by machine[16].

Working with the upstreams

kernoid is not a fork that follows Stalwart or Zitadel release by release. Code that kernoid takes over becomes part of kernoid and develops on its own. Fixes found along the way go back where the projects accept them:

[14] Own count of the `SPDX-License-Identifier` lines in the `.rs` files of `stalwartlabs/stalwart`, branch `main`, commit `aa47d8e`, 2026-10-22; licence texts in `LICENSES/`: https://github.com/stalwartlabs/stalwart/tree/main/LICENSES
[15] Zitadel, `LICENSING.md`, checked 2026-10-22: https://github.com/zitadel/zitadel/blob/main/LICENSING.md
[16] REUSE Specification, Free Software Foundation Europe, checked 2026-10-22: https://reuse.software/spec/

- **Zitadel** accepts contributions under Apache-2.0[17].
- **Stalwart** currently accepts only bug fixes and translations, only from contributors it has vouched for, only under its Fiduciary Contributor License Agreement, and no AI-generated code[18]. kernoid reports what it finds and offers fixes within these rules.

Security fixes in both projects concern kernoid as well: kernoid follows the security advisories of both and checks every fix against the code it took over. **[OPEN: contact with both projects before publication]**

Integrations

kernoid builds the core and leaves the rest to free software that already works: documents, video and chat. These services do not become part of kernoid; the operator runs them next to it, and they connect to kernoid through open standards. Where a service needs more than a standard, kernoid has an optional module for it (see Modules) – an organisation that uses no video conferencing never has to think about Jitsi.

How services connect

Need	Standard	kernoid's part
Login	OpenID Connect, SAML 2.0	identity provider: one login for every connected service
Accounts in other services	SCIM 2.0	accepts accounts from outside, and creates and removes accounts in connected services – leaving the organisation ends access everywhere in one step
Directory for older software	LDAP	directory that such software can read
Editing files in place	WOPI	file host that office software opens and saves files on

Any service that supports one of these standards can use kernoid without changes to either side.

Planned integrations

- **Documents:** Collabora Online and ONLYOFFICE Docs open and save files on a file host over WOPI[19]. kernoid's storage acts as that host, so documents are edited where they are stored, with kernoid's rights. The WOPI module comes before 1.0 (see Roadmap).
- **Video:** Jitsi Meet authenticates users internally, over LDAP or with signed tokens (JWT); its documentation names no OpenID Connect[20]. kernoid's Jitsi module issues these tokens for logged-in accounts – no extra service is needed – and adds a meeting to an event with one click.
- **Chat:** for Matrix, the Matrix Authentication Service accepts an upstream OpenID Connect provider, but no SAML[21]. kernoid is that provider.

Further integrations follow when someone needs them and tests them.

[17] Zitadel, `LICENSING.md`, checked 2026-10-22: https://github.com/zitadel/zitadel/blob/main/LICENSING.md

[18] Stalwart, `CONTRIBUTING.md`, checked 2026-10-22: https://github.com/stalwartlabs/stalwart/blob/main/CONTRIBUTING.md

[19] Collabora Online, `README.md` (integration with a host via a WOPI URL; MPL-2.0): https://github.com/CollaboraOnline/online.mirror/blob/main/README.md; ONLYOFFICE Docs API, "WOPI REST API", checked 2026-10-22: https://api.onlyoffice.com/docs/docs-api/using-wopi/wopi-rest-api/

[20] Jitsi Meet Handbook, "Self-Hosting Guide – Docker", section Authentication, checked 2026-10-22: https://jitsi.github.io/handbook/docs/devops-guide/devops-guide-docker/

[21] Matrix Authentication Service documentation, "Single sign-on", checked 2026-10-22: https://element-hq.github.io/matrix-authentication-service/setup/sso.html

DRAFT

Actions and hooks

Services that kernoid has no module for still take part in what happens in it – as separate services that talk to kernoid over the network, written in any language, maintained by anyone:

- **Actions** tell a service what happened – an account was created, a member left a group – signed, in order, and retried until delivered.
- **Hooks** let a service take part in a decision while it is made: whether a login may go ahead, whether a message is accepted, what an account receives in another system. kernoid waits for the answer only as long as configured and then follows the policy set for that hook: allow or deny.
- An integration describes itself in a **manifest**: the events it receives, the hooks it answers, the rights it needs. A tenant's administrator installs it and sees what it may do; it works with a service account limited to exactly that.
- What integrations see is a **public, versioned set of events**, separate from kernoid's internal events – kernoid can change its data model without breaking them.

An integration shares no code with kernoid, only the network interface **[OPEN: confirm what this means for the licences of integrations before publication]**. Actions and hooks follow before 1.0 (see Roadmap).

What an integration means

An integration is listed when it has been tested against a released version of kernoid and its setup is documented. There is no certification and no partner programme behind it; the setup guides are part of kernoid's documentation.

Governance and Funding

A falkOSS project

kernoid is published by **falkOSS – Falcon Lake Open Source Software gUG (haftungsbeschränkt) i. G.**, a non-profit company for free software in Falkensee near Berlin, currently in formation **[OPEN: state of incorporation and of the non-profit recognition at publication]**. falkOSS holds kernoid's name, logo, domains and GitHub organisation and funds its development. kernoid keeps its own identity; falkOSS appears as its publisher.

The technical direction lies with the project. A small group of maintainers decides; larger changes – to the protocol, the data model or the licences – are first published as a proposal (RFC) and discussed in the open. Whoever contributes well over time can become a maintainer.

Principles

Two principles of falkOSS apply to every falkOSS project and are part of its articles of association **[OPEN: until notarisation, a draft]**:

- **Everything included.** The free release is the whole product: one edition, no enterprise or pro edition, no licence keys, no subscriptions, no features released later for paying customers. Money is earned for work – operation, support, development – never for unlocking.
- **No telemetry.** kernoid sends no usage data anywhere. Exceptions are only what is technically necessary, what the user starts themselves, and what an operator explicitly opts into.

DRAFT

Licences

Part	Licence
Server	GNU AGPL-3.0-or-later; code taken over from Stalwart and Zitadel AGPL-3.0-only (see Building on Stalwart and Zitadel)
Terminal app	GNU AGPL-3.0-or-later
Web, desktop and mobile apps, client libraries	MIT
Protocol specification	CC BY 4.0
Documentation and this whitepaper	CC BY 4.0 – like the specification, so that text moves freely between them
Logo and icons (the artwork)	CC BY-ND 4.0 – to be passed on unchanged
Name and brand	not covered by the licences; held by falkOSS, which registers "kernoid" as a word mark once it is incorporated

Contributions are accepted under the Developer Certificate of Origin (`git commit -s`) – there is no contributor licence agreement, and nobody gets rights to the code that others do not have.

Funding

kernoid is funded through falkOSS, by:

- **donations** – tax-deductible once falkOSS is recognised as non-profit;
- **grants** for open digital infrastructure, such as those of the Sovereign Tech Agency[22] or NLnet's NGI Zero funds[23] **[OPEN: which programmes fit and accept a project at kernoid's stage]**;
- **work** for organisations that build on kernoid: support, development of features they need – which then go into the one free release.

The roadmap (see Roadmap) is planned with two to three full-time developers; budgets are part of the individual grant applications.

Managed service

falkOSS does not sell kernoid as a service. A **separate company** is planned to offer managed hosting **[OPEN: name, timing]**; any other provider can do the same. That company runs the same software under the same licence and has no rights that other providers lack.

falkOSS itself hosts kernoid only for non-profit purposes – and runs its own email on kernoid as soon as it is stable.

Roadmap

The order is fixed; the timing is an aim, not a promise – with fewer people or less money, it moves.

Phase	Version	Content
Pre-release	0.1 (released 2026-10-28)	identity and single sign-on, mail, calendars and contacts, files, the web app and a terminal app
	0.2	the core completed: public links, resumable uploads, LDAP, the Kubernetes operator; end-to-

[22] Sovereign Tech Agency, checked 2026-10-22: https://www.sovereign.tech
[23] NLnet, "Propose a project", checked 2026-10-22: https://nlnet.nl/propose/

Phase	Version	Content
		end encryption; virus scanning and Jitsi as the first modules
	further 0.x	WOPI, external key management, DNS automation, moving over; actions and hooks; the app for desktop and mobile; external security audit
Stable	1.0, once falkOSS is incorporated and holds the app stores' developer accounts	ready for production, with the app for desktop and mobile; falkOSS runs its own email, logins and files on it

Throughout the pre-release phase, kernoid's own apps are the **web app** and the **terminal app**; they are made as good as the core. The **app for desktop and mobile** is built alongside and released with 1.0 – publishing it in the app stores needs falkOSS as a legal entity with developer accounts. Everything else connects over the open protocols.

0.1 – the first pre-release

- tenants, accounts, groups, roles, projects – as events, with the audit trail as data
- login with MFA and passkeys; OpenID Connect and OAuth 2.0 for other applications; SAML 2.0 and OpenID Connect federation; SCIM 2.0
- mail: SMTP in and out, IMAP, JMAP, spam filtering, SPF, DKIM and DMARC
- calendars and contacts (CalDAV, CardDAV, JMAP); files with versions, sharing, team folders and WebDAV
- the web app with administration, the terminal app; one container image with Docker Compose, PostgreSQL and object storage

Pre-releases are for testing, not for production data.

0.2 – the core completed, the first modules

- public links to files, uploads that resume after a broken connection
- LDAP for programs that need it; the open points of mail (SRS, sending as a group, OAUTHBEARER, IMAP `SORT` and `THREAD`)
- running in separate roles across several instances, with the Kubernetes operator
- **end-to-end encryption** with OpenPGP – mail, vaults (encrypted folders), calendars and address books – in the web app and the terminal app (RFC 0006)
- **virus scanning** of incoming mail and uploaded files, and **Jitsi** – video meetings with kernoid's login – as the first modules
- what testing 0.1 turns up

Push notifications without any provider's account – Web Push for the web app, UnifiedPush for Android – follow in 0.x (RFC 0007).

Until stable

The further modules come in this order (see Modules):

1. **WOPI** – editing documents where they are stored, with Collabora Online or ONLYOFFICE Docs.
2. **External key management and DNS automation** – the main key held in an external key store; DNS records for mail (DKIM, MTA-STS and others) set through the APIs of DNS providers.
3. **Moving over** – from Microsoft 365 and Google Workspace, including accounts and groups from Active Directory, and from any other service over IMAP and, where offered, CalDAV and CardDAV (for example Apple iCloud or GMX).

DRAFT

Alongside:

- the app for desktop and mobile, with end-to-end encryption from its first version (see Apps and protocol)
- actions and hooks for integrations (see Integrations)
- stable data formats and a documented upgrade path between versions
- documentation for operators, including moving over from other systems
- external security audit, results published

1.0 – stable

Ready for production, as soon as falkOSS is incorporated and holds the developer accounts the app stores require. The app for desktop and mobile comes with it, and with it push notifications through Apple and Google – set up by each operator with their own keys or a relay of their choice; nothing is sent there by default. falkOSS moves its own email, logins and files to kernoid.

After 1.0

Without dates and in no fixed order: further integrations, an optional external search index.

Get Involved

kernoid is at its beginning. This is the moment when help shapes the most.

Developers

kernoid needs people who know or want to learn:

- **Rust** – async, networking, systems programming
- **identity protocols** – OpenID Connect, OAuth 2.0, SAML, SCIM, WebAuthn
- **mail and groupware protocols** – SMTP, IMAP, JMAP, CalDAV, CardDAV
- **distributed systems** – PostgreSQL, object storage, event sourcing
- **apps** – Flutter for desktop and mobile, TypeScript for the web
- **operation** – containers, Kubernetes, security

Every contribution is welcome – code, review, documentation, translations, testing. Contributions are accepted under the Developer Certificate of Origin.

Organisations

- **Try it:** test the pre-releases and tell us what is missing for your organisation.
- **Fund it:** donations and grants pay for development; commissioned work goes into the one free release.
- **Offer it:** run kernoid for your customers – the software is the same for everyone.

Public sector and funders

kernoid is free software for a core every administration needs. Public bodies can fund its development, test it in pilots and ask for open standards in procurement.

Contact

- **GitHub:** https://github.com/kernoid
- **Website:** https://kernoid.org; documentation: https://kernoid.dev

DRAFT

- **Email, chat: [OPEN]**
- **Publisher:** falkOSS – https://falkoss.org

Questions and corrections to this whitepaper are welcome as issues or pull requests in its repository: https://github.com/kernoid/whitepaper.

DRAFT